

MI FB API

1 Overview

The API of MStarFB is divided into the following 4 categories:

1.1. File operation class

Provides an interface for operating MStarFB. By calling these interfaces, overlays can be manipulated like files. These interfaces are standard interfaces provided by Linux itself, mainly including `open` , `close` , `write` , `read` , and so `lseek` on. These standard interfaces are not described in this document.

1.2. Video memory mapping class

Provides an interface for mapping physical video memory to user virtual memory space. These interfaces are standard interfaces provided by Linux itself, mainly including `mmap` , `munmap` etc. These standard interfaces are not described in this document.

1.3. Video memory control and status query class

An interface that allows setting properties such as pixel format and color depth. These interfaces are standard interfaces provided by Linux itself and are often used. This document will briefly describe it.

1.4. Interlayer effect control and state query class

MStarFB can manage multiple graphics overlays, and each layer can set properties such as alpha information and display area. These are new features of MStarFB relative to the Linux Framebuffer. This document will focus on that part. This part of the function is realized by extending the command parameters of the standard linux Framebuffer ioctl function.

ioctl function:

The user mode interface of MStarFB is embodied in the form of ioctl, and its form is as follows:

```
int ioctl(int fd,
unsigned long cmd,
.....
);
```

This function is a Linux standard interface with variable parameters. But in MStarFB, only 3 parameters are actually required. Therefore, its grammatical form is equivalent to:

```
int ioctl (int fd,
unsigned long cmd,
CMD_DATA_TYPE *cmddata);
```

Among them, CMD_DATA_TYPE changes with the change of parameter cmd. The detailed description of these three parameters is shown in Table 1-1.

Table 1-1 The three parameters of the ioctl function

parameter	describe	input
name parameter	describe	Output input
name Fd	Framebuffer device file descriptor, which is opened by	Output enter

calling the open function

Cmd	主要的 cmd (命令控制字) 如下： FBIOGET_VSCREENINFO [#21-fbioget_vscreeninfo]：获取屏幕可变信息 FBIOPUT_VSCREENINFO [#22-fbioput_vscreeninfo]：设置屏幕可变信息 FBIOGET_FSCREENINFO [#23-fbioget_fscreeninfo]：获取屏幕固定信息 FBIOPAN_DISPLAY [#24-fbiopan_display]：设置 PAN 显示 FBIOGET_SCREEN_LOCATION [#31-fbioget_screen_location]：获取叠加层显示区域 FBIOPUT_SCREEN_LOCATION [#32-fbioput_screen_location]：设置叠加层显示区域 FBIOGET_SHOW [#33-fbioget_show]：获取叠加层显示状态 FBIOPUT_SHOW [#34-fbioput_show]：设置叠加层显示状态 FBIOGET_GLOBAL_ALPHA [#35-fbioget_global_alpha]：获取叠加层 Alpha属性 FBIOPUT_GLOBAL_ALPHA [#36-fbioput_global_alpha]：设置叠加层 Alpha属性 FBIOGET_COLORKEY [#37-fbioget_colorkey]：获取叠加层的 Colorkey 属性 FBIOPUT_COLORKEY [#38-fbioput_colorkey]：设置叠加层的 Colorkey 属性 FBIOGET_DISPLAY_LAYER_ATTRIBUTES [#310-fbioget_displaylayer_attributes]：获取叠加层属性 FBIOPUT_DISPLAY_LAYER_ATTRIBUTES [#39-fbioput_displaylayer_attributes]：设置叠加层属性 FBIOGET_CURSOR_ATTRIBUTE [#41-fbioget_cursor_attribute]：获取鼠标层属性 FBIOPUT_CURSOR_ATTRIBUTE [#42-fbioput_cursor_attribute]：设置鼠标层属性	输入
cmddata	各 cmd 对应的数据类型分别是： 获取或设置屏幕可变信息：struct fb_var_screeninfo [#612-struct-fb_var_screeninfo] *类型 获取屏幕固定信息：struct fb_fix_screeninfo [#613-struct-fb_fix_screeninfo] *类型 设置 PAN 显示：struct fb_var_screeninfo [#612-struct-fb_var_screeninfo] *类型 获取或设置屏幕叠加层坐标原点：MI_FB_CursorAttr_t [#6211-mi_fb_cursorattr_t] *类型	输入输出
parameter name	获取或设置叠加层显示状态：MI_BOOL *类型 获取或设置叠加层 Alpha：MI_FB_GlobalAlpha_t [#623-mi_fb_globalalpha_t] *类型	input Output

获取或设置叠加层Colorkey: [MI_FB_ColorKey_t](#) [#624-mi_fb_colorkey_t]*类型
获取或设置叠加层属性:[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]*类型
获取或设置鼠标图层属性: [MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]*

2. 标准功能

2.1. FBIOGET_VSCREENINFO

- 功能

获取屏幕的可变信息。

- 语法

```
int ioctl (int fd,FBIOGET_VSCREENINFO,struct fb_var_screeninfo
*var);
```

- 描述

使用此接口获取屏幕的可变信息，主要包括分辨率和像素格式。
信息的详细描述请参见[struct fb_var_screeninfo](#) [#612-struct-fb_var_screeninfo]。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_VSCREENINFO [#21-fbioget_vscreeninfo]	ioctl 号	输入
var	可变信息结构体指针	输出

- 返回值
 - 0 成功。
 - -1 失败。

- 依赖
头文件：fb.h

- 相关接口

[FBIOPUT_VSCREENINFO](#) [#22-fbioput_vscreeninfo]

2.2. FBIOPUT_VSCREENINFO

- 功能
设置 Framebuffer 的屏幕分辨率和像素格式等。

- 语法

```
int ioctl (int fd, FBIOPUT_VSCREENINFO, structfb_var_screeninfo *var);
```

- 描述
使用此接口设置屏幕分辨率、像素格式。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOPUT_VSCREENINFO [#22-fbioput_vscreeninfo]	ioctl 号	输入
var	可变信息结构体指针	输入

- 返回值

- 0 成功。
- -1 失败
- 依赖

头文件：`fb.h`

- 注意

必须保证实际分辨率与偏移的和在虚拟分辨率范围内，否则系统会自动调整实际分辨率的大小让其在虚拟分辨率范围内。

- 相关接口

[FBIOGET_VSCREENINFO](#) [#21-fbioget_vscreeninfo]

2.3. FBIOGET_FSCREENINFO

- 功能

获取 `Framebuffer` 的固定信息。

- 语法

```
int ioctl (int fd, FBIOGET_FSCREENINFO, *fix);
```

- 描述

使用此接口获取 `Framebuffer` 固定信息，包括显存起始物理地址、显存大小和行间距等。信息的详细描述请参见[struct fb_fix_screeninfo](#) [#613-struct-fb_fix_screeninfo]

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_FSCREENINFO [#23-fbioget_fscreeninfo]	ioctl 号	输入
fix	固定信息结构体指针	输出

- 返回值
 - 0 成功
 - -1 失败
- 依赖

头文件：fb.h

2.4. FBIOPAN_DISPLAY

- 功能

设置从虚拟分辨率中的不同偏移处开始显示。
- 语法

```
int ioctl (int fd, FBIOPAN_DISPLAY, *var);
```

- 描述

使用此接口设置从虚拟分辨率中的不同偏移处开始显示，实际的分辨率不变。如图 2-1 所示：(xres_virtual, yres_virtual) 是虚拟分辨率，(xres, yres) 是实际显示的分辨率，(xoffset, yoffset) 是显示的偏移。
- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOPAN_DISPLAY [#24-fbiopan_display]	ioctl 号	输入
var	可变信息结构体指针	输入

- 返回值
 - 0 成功。
 - -1 失败

- 依赖

头文件：`fb.h`

- 注意

必须保证实际分辨率与偏移的和在虚拟分辨率范围内，否则设置不成功。

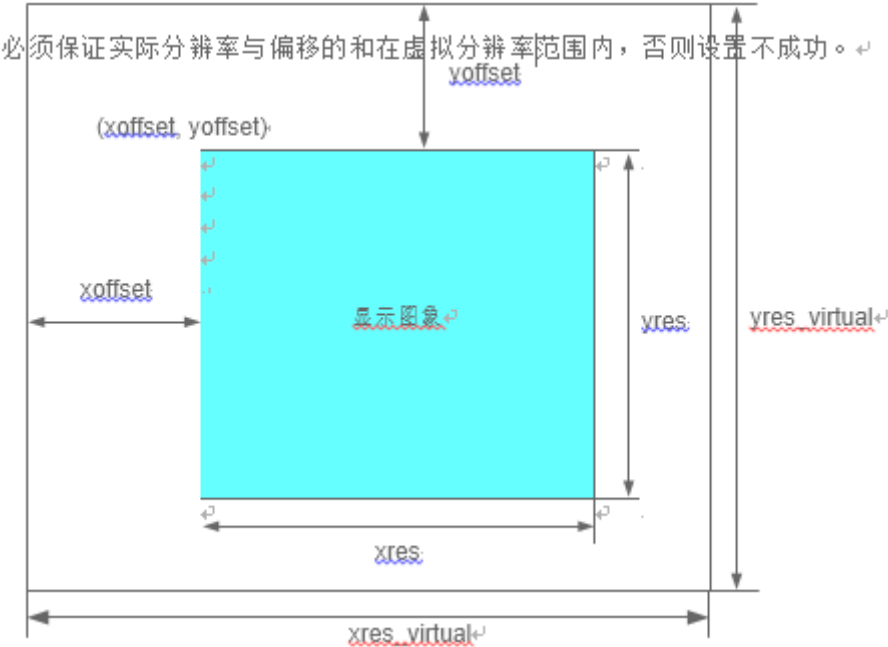


图2-1 设置从虚拟分辨率中的不同偏移处开始显示

3. 扩展功能

3.1. FBIOGET_SCREEN_LOCATION

- 功能

获取叠加层在屏幕上的显示区域

- 语法

```
int ioctl (int fd,FBIOGET_SCREEN_LOCATION,MI_FB_Rectangle_t*
pstRectangle);
```

- 描述

使用此接口获取叠加层在屏幕上的显示区域

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_SCREEN_LOCATION [#31-fbioget_screen_location]	ioctl 号	输入
pstRectangle	MI_FB_Rectangle_t [#625-mi_fb_rectangle_t]结构体指针，包含(x,y)坐标和width,height。表示叠加层在屏幕上的显示区域	输出

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

3.2. FBIOSET_SCREEN_LOCATION

- 功能

设置叠加层在屏幕上的显示区域

- 语法

```
int ioctl (int fd,FBIOSET_SCREEN_LOCATION,MI_FB_Rectangle_t*
pstRectangle);
```

- 描述

使用此接口设置叠加层在屏幕上的显示区域

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_SCREEN_LOCATION [#32-fbioset_screen_location]	ioctl 号	输入
pstRectangle	MI_FB_Rectangle_t [#625-mi_fb_rectangle_t]结构体指针，包含(x,y)坐标和width,height。表示叠加层在屏幕上的显示区域	输入

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

3.3. FBIOGET_SHOW

- 功能
获取当前叠加层的显示状态。
- 语法

```
int ioctl (int fd,FBIOGET_SHOW,MI_BOOL *bShow);
```

- 描述
使用此接口获取当前叠加层显示状态。
- 形参

参数名称	描述	输入/输出
Fd	Framebuffer 设备文件描述符	输入
F BIOGET_SHOW [#33-fbioget_show]	ioctl 号	输入
bShow	指示当前叠加层的状态： *bShow = MS_TRUE：当前叠加层处于显示状态 *bShow = MS_FALSE：当前叠加层处于隐藏状态	输出

- 返回值
 - 0 成功
 - -1 失败
- 依赖
头文件：mstarFb.h
- 相关接口

3.4. FBIOSET_SHOW

- 功能
显示或隐藏该叠加层。
- 语法

```
int ioctl (int fd,FBIOSET_SHOW,MI_BOOL *bShow);
```

- 描述
使用此接口设置叠加层显示状态：显示或隐藏。
- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_SHOW [#34-fbioset_show]	ioctl 号	输入
bShow	该叠加层的显示状态： *bShow = MS_TRUE：显示当前叠加层 *bShow = MS_FALSE：隐藏当前叠加层	输入

- 返回值
 - 0 成功
 - -1 失败
- 依赖
头文件：mstarFb.h
- 相关接口

3.5. FBIOGET_GLOBAL_ALPHA

- 功能

获取叠加层 Alpha 设置。

- 语法

```
int ioctl (int
fd,FBIOGET_GLOBAL_ALPHA,MI_FB_GlobalAlpha_t*pstAlpha);
```

- 描述

使用此接口获取当前叠加层的 Alpha 设置。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_GLOBAL_ALPHA [#35-fbioget_global_alpha]	ioctl 号	输入
pstAlpha	MI_FB_GlobalAlpha_t [#623- mi_fb_globalalpha_t] 结构体指针	输出

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

[FBIOSET_GLOBAL_ALPHA \[#36-fbioset_global_alpha\]](#)

3.6. FBIOSSET_GLOBAL_ALPHA

- 功能

设置叠加层的 Alpha。

- 语法

```
int ioctl (int
fd, FBIOSSET_GLOBAL_ALPHA, MI_FB_GlobalAlpha_t*pstAlpha);
```

- 描述

使用此接口设置当前叠加层的 Alpha 功能。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSSET_GLOBAL_ALPHA [#36-fbioset_global_alpha]	ioctl 号	输入
pstAlpha	MI_FB_GlobalAlpha_t [#623-mi_fb_globalalpha_t] 结构体指针	输入

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

[FBIOSGET_GLOBAL_ALPHA](#) [#35-fbioset_global_alpha]

3.7. FBIOSGET_COLORKEY

- 功能

获取叠加层的 colorkey。

- 语法

```
int ioctl (int fd,FBIOGET_COLORKEY,MI_FB_ColorKey_t*
pstColorKey);
```

- 描述

使用此接口获取叠加层的 colorkey。

- 形参

参数名称	描述	输入/输出
Fd	Framebuffer 设备文件描述符	输入
FBIOGET_COLORKEY [#37-fbioget_colorkey]	ioctl 号	输入
pstColorKey	MI_FB_ColorKey_t [#624-mi_fb_colorkey_t] 结构体指针	输出

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

[FBIOSET_COLORKEY](#) [#38-fbioset_colorkey]

3.8. FBIOSET_COLORKEY

- 功能

设置叠加层的 colorkey。

- 语法

```
int ioctl (int fd,FBIOSET_COLORKEY,MI_FB_ColorKey_t*
pstColorKey);
```

- 描述

使用此接口设置当前叠加层的 colorkey 功能。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_COLORKEY [#38-fbioset_colorkey]	ioctl 号	输入
pstColorKey	MI_FB_ColorKey_t [#624-mi_fb_colorkey_t] 结构体指针	输入

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

[FBIOGET_COLORKEY](#) [#37-fbioget_colorkey]

3.9. FBIOSET_DISPLAYLAYER_ATTRIBUTES

- 功能

设置图层信息。

• 语法

```
int ioctl (int fd,FBIOSET_DISPLAYLAYER_ATTRIBUTES,
MI_FB_DisplayLayerAttr_t* pstLayerInfo);
```

• 描述

此接口用于设置图层信息，包括图层在屏幕的显示区域、画布分辨率、显存分辨率、屏幕显示分辨率以及是否使能预乘。以上信息的更详细说明见 [MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]的描述。

• 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_DISPLAYLAYER_ATTRIBUTES [#39-fbioset_displaylayer_attributes]	ioctl 号	输入
pstLayerInfo	MI_FB_DisplayLayerAttr_t [#626-mi_fb_displaylayerattr_t]结构体指针	输出

• 返回值

- 0 成功
- -1 失败

• 依赖

头文件：mstarFb.h

• 相关接口

[FBIOSET_DISPLAYLAYER_ATTRIBUTES](#) [#39-fbioset_displaylayer_attributes]

3.10. FBIOGET_DISPLAYLAYER_ATTRIBUTES

- 功能
获取图层信息。
- 语法

```
int ioctl (int fd,  
  
FIOGET_DISPLAYLAYER_ATTRIBUTES,  
  
MI_FB_DisplayLayerAttr_t * pstLayerInfo);
```

- 描述
用于获取图层信息，包括图层在屏幕的显示区域、显存分辨率、屏幕显示分辨率以及是否使能预乘。
- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FIOGET_DISPLAYLAYER_ATTRIBUTES [#310-fbioget_displaylayer_attributes]	ioctl 号	输入
pstLayerInfo	图层信息结构体指针	输出

- 返回值
 - 0 成功
 - -1 失败
- 依赖
头文件：mstarFb.h
- 相关接口

[FIOGET_DISPLAYLAYER_ATTRIBUTES](#) [#310-fbioget_displaylayer_attributes]

4. 鼠标相关功能

4.1. FBIOGET_CURSOR_ATTRIBUTE

- 功能

获取鼠标图层信息。

- 语法

```
int ioctl (int fd,
           FBIOGET_CURSOR_ATTRIBUTE,
           MI_FB_CursorAttr_t*pstCursorAttr)
```

- 描述

获取鼠标图层信息，包括 **hotspot**、**位置**、**Alpha**、**Colorkey**、是否可见。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_CURSOR_ATTRIBUTE [#41-fbioget_cursor_attribute]	ioctl 号	输入
pstCursorAttr	MI_FB_CursorAttr_t [#6211-mi_fb_cursorattr_t]结构体指针	输出

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

[FBIOSET_CURSOR_ATTRIBUTE](#) [#42-fbioset_cursor_attribute]

4.2. FBIOSET_CURSOR_ATTRIBUTE

- 功能

设置鼠标图层信息。

- 语法

```
int ioctl (int fd,
           FBIOSET_CURSOR_ATTRIBUTE,
           MI_FB_CursorAttr_t *pstCursorAttr)
```

- 描述

设置鼠标图层信息, 包括hotspot、位置、Alpha、Colorkey、是否可见。

- 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_CURSOR_ATTRIBUTE [#42-fbioset_cursor_attribute]	ioctl 号	输入
pstCursorAttr	MI_FB_CursorAttr_t [#6211-mi_fb_cursorattr_t]结构体指针	输出

- 返回值

- 0 成功
- -1 失败

- 依赖

头文件：mstarFb.h

- 相关接口

`FBIOGET_CURSOR_ATTRIBUTE` [#41-fbioget_cursor_attribute]

5. 错误码

表 2-2 列出了当函数返回值小于 0 时有可能出现的所有错误码。这些错误码来自标准的 linux 错误码定义，详细内容请参见 linux 内核原码 `errno_base.h`。错误码可以通过打印 Linux 的标准错误码 `errno` 查看，或者用 `strerror(errno)`打印错误信息。

表2-2 错误码

错误代码	宏定义	描述
1	EPERM	不支持该操作
12	ENOMEM	内存不够
14	EFAULT	传入参数指针地址无效
22	EINVAL	传入参数无效

6. FB数据类型

6.1. 在标准中定义的数据类型

6.1.1. struct fb_bitfield

- 说明
位域信息，用于设置像素格式。
- 定义

```

struct fb_bitfield
{
    u32 offset; /* beginning of bitfield */

    u32 length; /* length of bitfield */

    u32 msb_right; /* != 0: Most significant bit is right */
};

```

- 成员

成员名称	描述	支持情况
offset	颜色分量起始比特位。	支持。
length	颜色分量所占比特长度。	支持。
msb_right	右边的比特是否为最高有效位。	只支持该位为 0，即最左边的 bit 为最高有效位。

6.1.2. struct fb_var_screeninfo

- 说明

可变的屏幕信息。

- 定义

```

struct fb_var_screeninfo
{
    u32 xres; /* visible resolution */

    u32 yres;

    u32 xres_virtual; /* virtual resolution */

    u32 yres_virtual;

    u32 xoffset; /* offset from virtual to visible */

```

```

u32 yoffset; /* resolution */

u32 bits_per_pixel; /* guess what */

u32 grayscale; /* != 0 Graylevels instead of colors */

struct fb_bitfield red; /* bitfield in fb mem if
true color, */ struct fb_bitfield green; /* else
only length is significant */ struct fb_bitfield
blue;

struct fb_bitfield transp; /* transparency */

u32 nonstd; /* != 0 Non standard pixel format */

u32 activate; /* see FB_ACTIVATE_* */

u32 height; /* height of picture in mm */

u32 width; /* width of picture in mm */

u32 accel_flags; /* (OBSOLETE) see fb_info.flags */

/* Timing: All values in pixclocks, except pixclock (of
course) */

u32 pixclock; /* pixel clock in ps (pico seconds) */

u32 left_margin; /* time from sync to picture */

u32 right_margin; /* time from picture to sync */

u32 upper_margin; /* time from sync to picture */

u32 lower_margin;

u32 hsync_len; /* length of horizontal sync */

u32 vsync_len; /* length of vertical sync */

u32 sync; /* see FB_SYNC_* */

u32 vmode; /* see FB_VMODE_* */

u32 rotate; /* angle we rotate counter clockwise */

u32 reserved[5]; /* Reserved for future compatibility */

```

```
};
```

• 成员

成员名称	描述	支持情况
xres	可见屏幕宽度（像素数）	支持
yres	可见屏幕高度（像素数）	支持
xres_virtual	虚拟屏幕宽度（显存中图像宽度），当该值小于 xres 时会修改xres，使 xres 值与该值相等	支持
yres_virtual	虚拟屏幕高度（显存中图像高度），当该值小于 yres 时会修改yres，使 yres 值与该值相等。结合xres_virtual，可以用来快速水平或垂直平移图像。	支持
xoffset	在 x 方向上的偏移像素数	支持，默认为 0
yoffset	在 y 方向上的偏移像素数	支持，默认为 0
bits_per_pixel	每个像素所占的比特数	支持
grayscale	灰度级	不支持，缺省值为 0，表示彩色
red	颜色分量中红色的位域信息	支持
green	颜色分量中绿色的位域信息	支持
blue	颜色分量中蓝色的位域信息	支持
transp	颜色分量中 alpha 分量的位域信息	支持
nonstd	是否为标准像素格式	不支持，缺省值为 0，表示支持标准像素格式。
成员名称 activate	描述 设置生效的时刻	支持情况 不支持，缺省值为

		FB_ACTIVATE_NOW，表示设置立刻生效。
height	屏幕高，单位为 mm	不支持，缺省值为-1。
width	屏幕宽，单位为 mm	不支持，缺省值为-1。
accel_flags	加速标志	不支持，缺省值为-1。
pixclock	显示一个点需要的时间，单位为ns	不支持，缺省值为-1。
left_margin right_margin hsync_len	分别是左消隐信号、右消隐信号、水平同步时长，这三个值之和等于水平回扫时间，单位为点时钟	不支持，缺省值为64
upper_margin lower_margin vsync_len	分别是上消隐信号、下消隐信号、垂直同步时长，这三个值之和等于垂直回扫时间，单位为点时钟。	不支持，缺省值为-1。

| sync | 同步信号方式 | 不支持，缺省值为-1。 || vmode | 扫描模式 | 不支持，缺省值为-1。 || rotate | 顺时针旋转的角度 | 不支持，缺省值为 0，表示无旋转。 |

- 注意事项

高清设备图形层默认的分辩率为 1280x720；标清设备图形层默认的分辩率为 720x576，鼠标层默认的分辩率为 128x128。像素格式为 ARGB1555。

- 相关数据类型及接口

[struct fb_bitfield](#) [#611-struct-fb_bitfield]

[FBIOGET_VSCREENINFO](#) [#21-fbioget_vscreeninfo]

[FBIOPUT_VSCREENINFO](#) [#22-fbioput_vscreeninfo]

6.1.3. struct fb_fix_screeninfo

- 说明

固定的屏幕信息。

- 定义

```
struct fb_fix_screeninfo
{
    char id[16]; /* identification string eg "TT Builtin" */
    unsigned long smem_start; /* Start of frame buffer mem
    (physical
    address) */

    u32 smem_len; /* Length of frame buffer mem */

    u32 type; /* see FB_TYPE_* */

    u32 type_aux; /* Interleave for interleaved Planes */

    u32 visual; /* see FB_VISUAL_* */

    u16 xpanstep; /* zero if no hardware panning */

    u16 ypanstep; /* zero if no hardware panning */

    u16 ywrapstep; /* zero if no hardware ywrap */

    u32 line_length; /* length of a line in bytes */ unsigned
long mmio_start;
    /* Start of Memory Mapped I/O (physical
    address) */

    u32 mmio_len; /* Length of Memory Mapped I/O */

    u32 accel; /* Indicate to driver which specific chip/card
we have */

    u16 reserved[3]; /* Reserved for future compatibility */
};
```

- 成员

成员名称	描述	支持情况
成员名称	描述	支持情况

id	设备驱动名称。	支持。
smem_start	显存起始物理地址。	支持。
smem_len	显存大小。	支持。
type	显卡类型。	固定为 FB_TYPE_PACKED_PIXELS， 表示像素值紧密排列。
type_aux	附加类型。	不支持，在 FB_TYPE_PACKED_PIXELS 显卡类型下无含义。
visual	色彩模式。	不支持，默认为 FB_TYPE_PACKED_PIXELS 显卡类型下无含义。
xpanstep	支持水平方向上的 PAN 显示： 0：不支持。 非 0：支持，此时该值用于表示在水平方向上每步进的像素值。	固定为 1。
ypanstep	支持垂直方向上的 PAN 显示： 0：不支持。 非 0：支持，此时该值用于表示在垂直方向上每步进的像素值。	固定为 1。
ywrapstep	该方式类似于 ypanstep，不同之处在于：当其显示到底部时，能回到显存的开始处进行显示。	不支持，默认为 0。
line_length	每行字节数。	支持。
mmio_start	显存映射 I/O 首地址。	不支持，默认为 0。
mmio_len	显存映射 I/O 长度。	不支持，默认为 0。
accel	显示所支持的硬件加速设备。	不支持，默认为 FB_ACCEL_NONE，无加速设备。
reserved	保留。	不支持，缺省值为 0。

- 相关数据类型及接口

[FBIOGET_FSCREENINFO](#) [#23-fbioget_fscreeninfo]

6.2. 扩展的数据类型

6.2.1. MI_FB_ColorFmt_e

- 说明

MStarFB 支持的像素格式集合。

- 定义

```
typedef enum
{
    E_MI_FB_COLOR_FMT_RGB565 = 1,
    E_MI_FB_COLOR_FMT_ARGB4444 = 2,
    E_MI_FB_COLOR_FMT_ARGB8888 = 5,
    E_MI_FB_COLOR_FMT_ARGB1555 = 6,
    E_MI_FB_COLOR_FMT_YUV422 = 9,
    /// Invalid color format.
    E_MI_FB_COLOR_FMT_INVALID = 12,
}MI_FB_ColorFmt_e;
```

- 成员

成员名称	描述
E_MI_FB_COLOR_FMT_RGB565	RGB565 格式。
E_MI_FB_COLOR_FMT_ARGB4444	ARGB444 格式。
E_MI_FB_COLOR_FMT_ARGB8888	ARGB8888
E_MI_FB_COLOR_FMT_ARGB1555	ARGB1555
E_MI_FB_COLOR_FMT_YUV422	YUV422
E_MI_FB_COLOR_FMT_INVALID	无效像素格式

- 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]

6.2.2. MI_FB_DisplayLayerAttrMaskbit_e

- 说明

标识[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]结构体内哪些成员有更新

- 定义

```
typedef enum
{
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS = 0x1,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE = 0x2,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_BUFFER_SIZE = 0x4,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE = 0x8,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_PREMUL = 0x10,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_COLOR_FMB = 0x20 .
}
```

```

E_MI_FB_DISPLAYLAYER_ATTR_MASK_OUTPUT_COLORSPACE = 0x40,

E_MI_FB_DISPLAYLAYER_ATTR_MASK_DST_DISP = 0x80,

} MI_FB_DisplayLayerAttrMaskbit_e;

```

- 成员

成员名称	描述
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS	叠加层位置发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE	叠加层在屏幕上的显示区域的宽度或高度发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_BUFFER_SIZE	显示分辨率发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE	屏幕分辨率发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_PREMUL	是否预乘法属性发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_COLOR_FMB	FB的Color format属性发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_OUTPUT_COLORSPACE	叠加层输出的Color Space发生变化
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DST_DISP	叠加层输出到display chain目标模组发生变化

- 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

6.2.3. MI_FB_GlobalAlpha_t

- 说明

Alpha 信息结构体

- 定义

```
typedef struct MI_FB_GlobalAlpha_s
{
    MI_BOOL bAlphaEnable; /* alpha enable flag */

    MI_BOOL bAlphaChannel; /* alpha channel enable flag */

    MI_BOOL bAlphaInvert; /* alpha invert flag */

    MI_U8 u8Alpha0; /* alpha0 value */

    MI_U8 u8Alpha1; /* alpha1 value */

    MI_U8 u8GlobalAlpha; /* global alpha value */ MI_U8
    u8Reserved;
} MI_FB_GlobalAlpha_t;
```

- 成员

成员名称	描述
bAlphaEnable	Alpha 叠加使能，默认为 1。
bAlphaChannel	Alpha 通道使能，默认为 0。
bAlphaInvert	Alpha 反转标记，用于argb1555下，0为不透明的情况。
u8Alpha0	Alpha0 值，范围 0~255，默认为 255。在 RGB1:5:5:5 格式下，当最高位为 0 时，选择该值作为 Alpha 叠加的 Alpha 值。
u8Alpha1	Alpha1 值，范围 0~255，默认为 255。在 RGB1:5:5:5 格式下，当最高位为 1 时，选择该值作为 Alpha 叠加的 Alpha 值。
u8GlobalAlpha	全局 Alpha 值，范围为 0~255，默认为 255。在 Alpha 通道使能时起作用。
u8Reserved	保留

- 注意事项
 - Alpha0 值只对 cursor 生效，UI 的 alpha0 恒为 0。
 - 只有在 Alpha 叠加使能的情况下才进行 Alpha 叠加，否则处于上层的叠加层将覆盖下层的叠加层。
 - 叠加 Alpha 值的计算公式有以下几种情况：
 - 当 Alpha 通道使能时，全局 Alpha 参与叠加。
 - 对于不支持全局 Alpha 和像素 Alpha 叠加的芯片，叠加 Alpha 值的计算公式如下所示： $\alpha = u * 8 * GlobalAlpha$
 - 对于支持全局 Alpha 和像素 Alpha 叠加的芯片，叠加 Alpha 值的计算公式如下所示： $\alpha = u * 8 * GlobalAlpha * a_{pixel}$
 - 当 Alpha 通道不使能时，叠加 Alpha 值等于像素 Alpha 值，即： $\alpha = a_{pixel}$
 - 相关数据类型及接口

[FBIOGET_GLOBAL_ALPHA](#) [#35-fbioget_global_alpha]

[FBIOSET_GLOBAL_ALPHA](#) [#36-fbioset_global_alpha]

6.2.4. MI_FB_ColorKey_t

- 说明
Colorkey信息 结构体，用于 colorkey 的属性设置。
- 定义

```
typedef struct MI_FB_ColorKey_s
{
    MI_BOOL bKeyEnable;

    MI_U8 u8Red;

    MI_U8 u8Green;

    MI_U8 u8Blue;
} MI_FB_ColorKey_t
```


- 成员

成员	描述
bKeyEnable	Colorkey 是否使能标识。 TRUE：使能； FALSE：不使能。
u8Red	Colorkey的R分量数值
u8Green	Colorkey的G分量数值
u8Blue	Colorkey的B分量数值

- 相关数据类型及接口

[FBIOGET_COLORKEY](#) [#37-fbioget_colorkey]

[FBIOSET_COLORKEY](#) [#38-fbioset_colorkey]

6.2.5. MI_FB_Rectangle_t

- 说明

[MI_FB_Rectangle_t](#) [#625-mi_fb_rectangle_t]结构体，用于描述叠加层的显示区域

- 定义

```
typedef struct MI_FB_Rectangle_s
{
    MI_U32  u32Xpos;
    MI_U32  u32Ypos;
    MI_U32  u32Width;
    MI_U32  u32Height;
}MI_FB_Rectangle_t;
```

- 成员

成员	描述
u32Xpos	左上角x坐标
u32Ypos	左上角y坐标
u32Width	宽度
u32Height	高度

- 相关数据类型及接口

[FBIOGET_SCREEN_LOCATION](#) [#31-fbioget_screen_location]

[FBIOSET_SCREEN_LOCATION](#) [#32-fbioset_screen_location]

6.2.6. MI_FB_DisplayLayerAttr_t

- 说明

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]结构体，描述叠加层的属性信息。

- 定义

```
typedef struct MI_FB_DisplayLayerAttr_s
{
    MI_U32 u32Xpos; /**the x pos of origin point in screen*/
    MI_U32 u32YPos; /**the y pos of origin point in screen*/
    MI_U32 u32dstWidth; /**display buffer dest with in screen*/
    MI_U32 u32dstHeight; /**display buffer dest hight in
screen*/
    MI_U32 u32DisplayWidth; /**the width of display buf in fb
*/
    MI_U32 u32DisplayHeight; /**the height of display buf in
fb. */
    MI_U32 u32ScreenWidth; /**the width of screen */
}
```

```

    MI_U32 u32ScreenHeight; /** the height of screen */

    MI_BOOL bPreMul; /**the data drawn in buffer whether is
    premultiply alpha
    or not*/

    MI_FB_ColorFmt_e eFbColorFmt; /**the color format of
    framebuffer*/

    MI_FB_OutputColorSpace_e eFbOutputColorSpace; /**output
    color space*/

    MI_FB_DstDisplayplane_e eFbDestDisplayPlane; /**destination
    displayplane*/

    MI_U32 u32SetAttrMask; /** display attribute modify mask*/

}MI_FB_DisplayLayerAttr_t;

```

- 成员

成员	描述
u32Xpos	图层在屏幕上的原点横坐标。
u32YPos	图层在屏幕上的原点纵坐标。
u32dstWidth	图层在屏幕上的显示宽度
u32dstHeight	图层在屏幕上的显示高度
u32DisplayWidth	显存分辨率的宽。
u32DisplayHeight	显存分辨率的高。
u32ScreenWidth	屏幕显示分辨率的宽。
u32ScreenHeight	屏幕显示分辨率的高。
bPreMul	FB 中的数据是否为预乘数据。
eFbColorFmt	FB的像素格式。
u32SetAttrMask	设置图层信息时参数修改掩码位。
eFbOutputColorSpace	设置叠加层输出的Color space
eFbDestDisplayPlane	设置叠加层输出到display chain的目的模组。默认为E_MI_FB_DST_OP0。

- 相关数据类型及接口

[MI_FB_DisplayLayerAttrMaskbit_e](#) [#622-mi_fb_displaylayerattrmaskbit_e]

[MI_FB_ColorFmt_e](#) [#621-mi_fb_colorfmt_e]

[MI_FB_DstDisplayplane_e](#) [#628-mi_fb_dstdisplayplane_e]

[MI_FB_OutputColorSpace_e](#) [#627-mi_fb_outputcolorspace_e]

6.2.7. MI_FB_OutputColorSpace_e

- 说明

MStarFB叠加层输出的Color Space。

- 定义

```
typedef enum
{
    E_MI_FB_OUTPUT_RGB = 0,
    E_MI_FB_OUTPUT_YUV = 1
}MI_FB_OutputColorSpace_e
```

- 成员

成员	描述
E_MI_FB_OUTPUT_RGB	RGB ColorSpace
E_MI_FB_OUTPUT_YUV	YUV ColorSpace

- 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

6.2.8. MI_FB_DstDisplayplane_e

- 说明

MStarFB叠加层输出到display chain的目的模组。通过配置不同的Display chain，可以选择叠加层参与的图像调整过程。

- 定义

```
typedef enum
{
    E_MI_FB_DST_OP0 = 0,
    E_MI_FB_DST_OP1 = 1,
    E_MI_FB_DST_INVALID
```

```
}MI_FB_DstDisplayplane_e
```

- 成员

成员	描述
E_MI_FB_DST_OP0	OP path
E_MI_FB_DST_OP1	OP1 path
E_MI_FB_DST_INVALID	MIXER2OP1

- 注意事项

此接口只在Ikayaki系列的芯片下有效。

- 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

6.2.9. MI_FB_CursorAttrMaskbit_e

- 说明

描述[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]结构体哪些属性发生变化

- 定义

```
typedef enum
{
    E_MI_FB_CURSOR_ATTR_MASK_ICON = 0x1,
    E_MI_FB_CURSOR_ATTR_MASK_POS = 0x2,
    E_MI_FB_CURSOR_ATTR_MASK_ALPHA = 0x4,
    E_MI_FB_CURSOR_ATTR_SHOW = 0x8,
    E_MI_FB_CURSOR_ATTR_HIDE = 0x10,
    E_MI_FB_CURSOR_ATTR_MASK = 0x1F
}
```

```
} MI_FB_CursorAttrMaskbit_e;
```

- 成员

成员	描述
E_MI_FB_CURSOR_ATTR_MASK_ICON	鼠标图标发生变化
E_MI_FB_CURSOR_ATTR_MASK_POS	鼠标位置发生变化
E_MI_FB_CURSOR_ATTR_MASK_ALPHA	Alpha发生变化
E_MI_FB_CURSOR_ATTR_SHOW	显示鼠标
E_MI_FB_CURSOR_ATTR_MASK_HIDE	隐藏鼠标
E_MI_FB_CURSOR_ATTR_MASK	用于标识鼠标所有可设置属性

- 相关数据类型及接口

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]

6.2.10. MI_FB_CursorImage_t

- 说明

[MI_FB_CursorImage_t](#) [#6210-mi_fb_cursorimage_t]结构体,描述鼠标图标数据信息。

- 定义

```
typedef struct MI_FB_CursorImage_s
{
    MI_U32 u32Width; /**width, unit pixel*/
    MI_U32 u32Height; /**Height, unit pixel*/
    MI_U32 u32Pitch; /**Pitch, unit pixel*/
    MI_FB_ColorFmt_e eColorFmt; /**Color format*/
}
```

```
const char* data;  
  
}MI_FB_CursorImage_t;
```

- 成员

成员	描述
u32Width	图标宽度
u32Height	图标高度
u32Pitch	pitch
eColorFmt	图标数据像素格式
data	图标数据

- 注意事项

图标的宽度和高度的最大值为128。

- 相关数据类型及接口

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]

[MI_FB_ColorFmt_e](#) [#621-mi_fb_colorfmt_e]

6.2.11. MI_FB_CursorAttr_t

- 说明

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]结构体,描述鼠标图层信息

- 定义

```
typedef struct MI_FB_CursorAttr_s  
{  
  
    MI_U32 u32XPos;  
  
    MI_U32 u32YPos;
```



```

MI_U32 u32HotSpotX;

MI_U32 u32HotSpotY;

MI_FB_GlobalAlpha_t stAlpha;

MI_FB_ColorKey_t stColorKey;

MI_BOOL bShown;

MI_FB_CursorImage_t stCursorImageInfo;

MS_U16 u16CursorAttrMask;

}MI_FB_CursorAttr_t;

```

- 成员

成员	描述
u32XPos	鼠标位置横坐标
u32YPos	鼠标位置纵坐标
u32HotSpotX	鼠标热点横坐标
u32HotSpotY	鼠标热点纵坐标
stAlpha	MI_FB_GlobalAlpha_t [#623-mi_fb_globalalpha_t] 结构体,鼠标图层alpha信息
stColorKey	MI_FB_ColorKey_t [#624-mi_fb_colorkey_t]结构体，鼠标图层colorkey信息
bShown	鼠标是否可见 TRUE表示可见，FALSE表示不可见
MI_FB_CursorImage_t [#6210-mi_fb_cursorimage_t]	图标数据信息。
u16CursorAttrMask	设置鼠标图层参数修改掩码位

- 相关数据类型及接口

[MI_FB_CursorAttrMaskbit_e](#) [#629-mi_fb_cursorattrmaskbit_e]

[MI_FB_CursorImage_t](#) [#6210-mi_fb_cursorimage_t]

7. PROCFS介绍

7.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_fb0

#####OnDumpOSDAttr#####

Framebuffer id = MStar FB0
xres=1280, yres=720
xres_virtual=1280, yres_virtual=1440
xoffset=0,yoffset=0
fix.line_length=0xa00 Bytes
fix.smem_start=0xf800000
Memory Size=0x400000 Bytes
Gop ID=0
Gwin ID=0
Open Count=1
Visible State=0
MIU Sel=0
ColorSpace=YUV
ColorFomrmat=ARGB1555
StretchWindow Pos[0,0]
StretchWindow Src[1280,720],StretchWindow Dst[1920,1080]
Gwin Pos[0,0]
Gwin Size[1280,720]
Gwin PhyAddr=0x0
ColorKey Enable=0
ColorKey Val=[0,0,0]
Enable Alpha Blend=1
Enalbe Multi Alpha=0
Global Alpha Val=0x0
Alpha0=0x0,Alpha1=0x0,(Only for ARGB1555)
GOP Hstart=192
```

```

Current TimingWidth=1920, TimingWidth=1080, hstar=192
#####end#####

#####OnDumpHwcursorAttr#####

Cursor Gop ID=4
Cursor Gwin ID=6
Cursor MIU Sel=0
Cursor PhyAddr=0x166000
Cursor ColorFmt=ARGB8888
Cursor Icon Width=44, Height=56
Cursor HotSpot[18, 9]
Cursor request pos[712, 207]
Cursor Visible=1
Cursor Gwin Pos[694, 198]
Cursor Gwin Size[44, 56]
Cursor Gwin Pitch=0x200 Bytes
Cursor StretchWindow pos[0, 0]
Cursor StretchWin Src[1280, 720], Dst[1920, 1080]
Cursor ColorKey Enable=0
Cursor ColorKey Value=[0xff, 0xff, 0xff]
#####OnDumpHwcursorAttr#####

```

- 调试信息分析

记录当前Fbdev && Hwcursor的使用状况以及OSD图层属性、OSD device属性、Hwcursor属性，可以动态地获取到这些信息，方便调试和测试。

- 参数说明

参数		描述
OSD info	Framebuffer id	Framebuffer设备ID
	xres	可见分辨率宽度
	yres	可见分辨率高度
	xres_virtual	虚拟分辨率的宽度
	yres_virtual	虚拟分辨率的高度
	xoffset	可见分辨率到虚拟分辨率的x偏移量

	yoffset	可见分辨率到虚拟分辨率的y偏移量
	fix.line_length	虚拟分辨率每行的字节数
	fix.smem_start	帧缓冲内起始地址
	Memory Size	帧缓冲内存长度，单位字节数
OSD device info	Gop ID	图层硬件ID
	Gwin ID	图层硬件显示窗口ID
	Open Count	该图形层打开次数。在用户调用open()时增1；调用close()时减1
	Visible State	该图层显示状态。 取值:{1表示可见，0表示不可见} 用户显示调用FBIOSET_SHOW去设置图层可见状态。
	MIU Sel	保存帧缓冲数据的物理内存编号。 对于MSR620设备该值为0
	ColorSpace	该图形层输出的颜色空间。 取值:{YUV表示输出YUV color space;RGB表示输出RGB color space}。默认为YUV
	ColorFomrmat	图形层格式 取值:{ARGB1555,ARGB4444,ARGB8888,RGB565}。 用户设置可变屏幕信息中的格式项后更新
	StretchWindow Pos	图形层在显示设备上的起始位置。 单位:像素 默认为0,用户可调用FBIOSET_SCREEN_LOCATION去设置
	StretchWindow Src	图形层原始分辨率的宽度和高度。 单位:像素 通过硬件放大功能放大到当前timing。用户设置可变屏幕信息中的可见分辨率后更新
	StretchWindow Dst	图形层在显示设备上的宽度和高度。默认设置为输出timing大小。可以通过FBIOSET_SCREEN_LOCATION

去设置

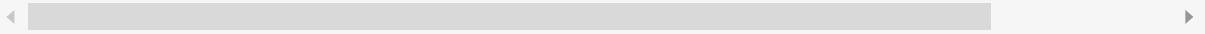
Gwin Pos	图形层显示窗口的起始位置 单位:像素 对于OSD图层来说,该值总是为(0,0)
Gwin Size	图形层显示窗口的宽度、高度。 单位:像素 对于OSD图层来说，该值总是被设置为可见分辨率的宽度、高度。用户设置可变屏幕信息中的可见分辨率后更新。
Gwin PhyAddr	图形层当前显示数据的物理地址
ColorKey Enable	图形层ColorKey是否使能 取值:{0表示Disable ColorKey; 1表示Enable ColorKey} 用户设置可通过FBIOSET_COLORKEY去设置
ColorKey Val	图形层ColorKey的RED,GREEN,BLUE通道数值 取值:每个通道的取值范围从0-255。ColorKey val的数值用RGB888格式表示 用户可通过FBIOSET_COLORKEY去设置后更新
Enable Alpha Blend	图形层alpha是否使能。 取值:{0表示否，1表示是}, 默认为1。 用户可通过FBIOSET_GLOBAL_ALPHA去设置后更新。 该项关闭，则像素alpha配置不生效。该项开启且Enable Multi Alpha关闭,仅像素alpha生效(对于ARGB1555格式,Alpha bit为1的像素使用Global Alpha作为像素alpha,Alpha bit为0的像素使用0xff作为像素alpha)。当该项目开启且Enable Multi Alpha生效时,则像素alpha和Global Alpha都生效
Enable Multi Alpha	全局alpha是否生效开关 取值{0时，1否}默认值为0 Enable Multi Alpha使能，Global Alpha便生效
Global Alpha Val	全局Alpha
Alpha0	ARGB1555格式时，当最高位为0时,选择Alpha0作为Alpha混合的的Alpha数值 取值: 0~255默认为0

	Alpha1	ARGB1555格式时，当最高位为1时，选择Alpha1作为Alpha混合的Alpha数值 取值: 0~255默认为0
	GOP Hstart	图层硬件向对于显示窗口的偏移。 取值:该值和当前输出timing相关，当输出timing发生变化时，通过FBIOSET_DISPLAYLAYER_ATTRIBUTES并将输入参数MI_FB_DisplayLayerAttr_s的u32SetAttrMask数值包含E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE mask位来通知图层驱动输出timing发生变化。图层驱动会根据当前timing设置相应的hstart数值
	Current TimingWidth,	当前输出timing的宽度、高度以及当前timing下应该给图形层硬件设置的像对于显示窗口的偏移。该项与GOP Hstart配合使用，用于检查设置给硬件图层的hstart数值是否和当前timing一致。
	Current TimingHeight,	
	hstar	
Hwcursor info	Cursor Gop ID	鼠标图层硬件ID
	Cursor Gwin ID	鼠标图层硬件显示窗口ID
	Cursor MIU Sel	用于保存鼠标图标数据的物理内存编号。对于MSR620设备该值为0
	Cursor PhyAddr	鼠标层当前显示数据的物理地址
	Cursor ColorFmt	鼠标图层格式 值:{ARGB1555,ARGB4444,ARGB8888,RGB565}。 用户可通过FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor Icon Width	鼠标图标的宽度、高度。 单位：像素 户可通过FBIOSET_CURSOR_ATTRIBUTE设置
	Cursor HotSpot	鼠标图标的热点信息 单位:像素 热点信息通常和鼠标图标相关，用户可通过FBIOSET_CURSOR_ATTRIBUTE设置

Cursor request pos	鼠标图标的位置信息 单位: 像素 用户通过FBIOSET_CURSOR_ATTRIBUTE设置
Cursor Visible	鼠标图层是否可见 取值:{0,表示不可见, 1表示可见} 用户通过FBIOSET_CURSOR_ATTRIBUTE设置
Cursor Gwin Pos	鼠标图层显示窗口的起始位置 单位:像素
Cursor Gwin Pitch	鼠标图层每行的字节数。 对于鼠标图层来说, 该值固定为0x200
Curosr StretchWindow pos	鼠标图层在显示设备上的起始位置。 单位:像素
Cursor StretchWin Src	鼠标图层原始大小 单位:像素。该值通常和OSD的显示分辨率一致。
Cursor StretchWin Dst	鼠标图层在显示设备上的宽度、高度 该值和输出timing一致。
Cursor ColorKey Enable	鼠标图层ColorKey是否使能 取值:{0表示Disable ColorKey; 1表示Enable ColorKey}。 用户通过FBIOSET_CURSOR_ATTRIBUTE设置
Cursor ColorKey Value	鼠标图层ColorKey的RED, GREEN, BLUE通道数值 取值:每个通道的取值范围从0-255。ColorKey val的数 值用RGB888格式表示 用户可通过FBIOSET_CURSOR_ATTRIBUTE去设置后更 新

7.2. echo

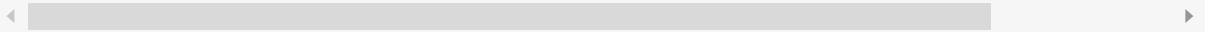
功能	打开或关闭指定的图层
命令	echo GUI_SHOW [LayerID] [Status] > /proc/mi_modules/mi_fb0
参数说明	[LayerID] 图层号 [Status] on: 打开; off: 关闭
举例	echo GUI_SHOW 0 on > /proc/mi_modules/mi_fb0 显示图层fb0 echo GUI_SHOW 0 off > /proc/mi_modules/mi_fb0 关闭图层fb0



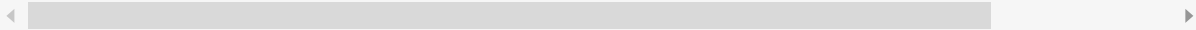
功能	打开或关闭硬件鼠标
命令	echo CURSOR_SHOW [Status] > /proc/mi_modules/mi_fb0
参数说明	[Status] on: 打开; off: 关闭
举例	echo CURSOR_SHOW on > /proc/mi_modules/mi_fb0 显示鼠标图标 echo CURSOR_SHOW off > /proc/mi_modules/mi_fb0 关闭鼠标图标



功能	为指定的图层设置Colorkey
命令	echo GUI_SET_CLRKEY [LayerID] [Red] [Green] [Blue] > /proc/mi_modules/mi_fb0
参数说明	[LayerID] 图层号 [Red] 红色分量 以16进制,ARGB888 格式表示 [Green] 绿色分量 以16进制,ARGB888 格式表示 [Blue] 蓝色分量 以16进制,ARGB888 格式表示
举例	echo GUI_SET_CLRKEY 0 ff 00 00 > /proc/mi_modules/mi_fb0 设置fb0的Colorkey为红色



功能	Dump当前正在显示的鼠标图标
命令	echo CURSOR_DUMP [Path] >/proc/mi_modules/mi_fb0
参数说明	[Path] dump 的路径
举例	echo CURSOR_DUMP /mnt > /proc/mi_modules/mi_fb0 dump cursor Icon到/mnt路径，文件名为CursorData.raw，用7yuv打开，宽高设置为按照cursor icon的格式设置即可显示Rawdata



功能	分别为ARGB1555格式的alpha bit0,alpha bi1设置pixel alpha数值。 因为目前msr620不支持这个feature，暂时未实现
命令	echo GUI_SETALPHA_ARGB1555 [LayerID] [Alpha0] [Alpha1]> /proc/mi_modules/mi
参数说明	[LayerID] 图层号 [Alpha0] alpha0 的数值 [Alpha1] alpha1 的数值
举例	无

